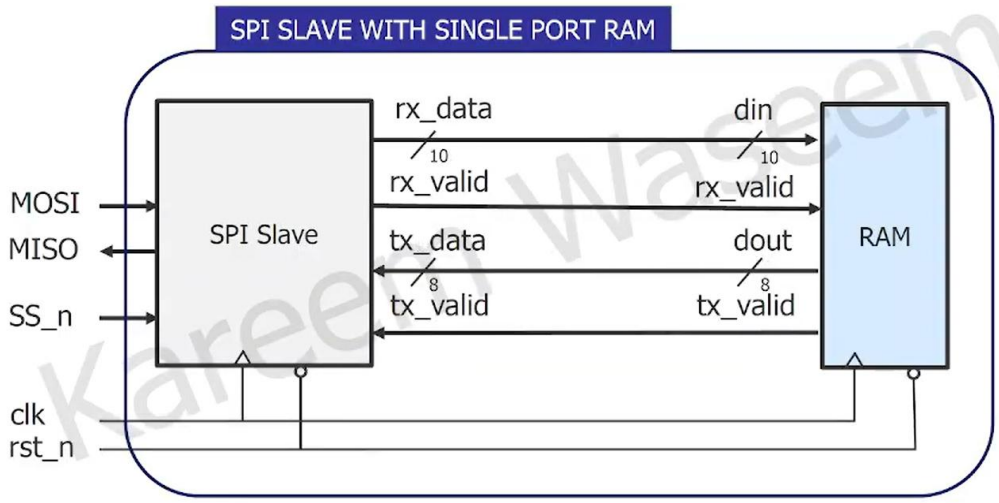
SPI Slave with Single Port RAM



Team Name: Byte Bandits

Supervised by : ENG/ Kareem waseem

Prepared by :

Name	Gmail
Nour Haytham	nourhaytham04@gmail.com
Ahmed Abdelrahman Abdalrazeq	m.ahmed.mohamed.al.al@gmail.com

Amr Alhassan Yacoob Hamed	hamr34357@gmail.com

1. RTL code:

- **SPI:**

```
module spi #(

    parameter IDLE = 3'b000,
    parameter CHK_CMD = 3'b001,
    parameter WRITE = 3'b010,
    parameter READ_ADD = 3'b011,
    parameter READ_DATA = 3'b100
)(
    input MOSI, SS_n, clk, rst_n,
    input tx_valid,
    input [7:0] tx_data,
    output reg rx_valid,
    output reg [9:0] rx_data,
    output reg MISO

);

reg [2:0] cs, ns;
reg add_exist;
reg [3:0] counter_in;
reg [2:0] counter_out;
reg [7:0] tx_reg;
reg start_out;

// State memory
always @(posedge clk ) begin
    if(!rst_n)
       cs <= IDLE;
    else
        cs <= ns;
end

// Next state logic
always @(*) begin
    case (cs)
        IDLE: ns = (SS_n == 0) ? CHK_CMD : IDLE;
```

```
        CHK_CMD: begin
            if (SS_n == 1)
                ns = IDLE;
            else if (MOSI == 0)
                ns = WRITE;
            else
                ns = (add_exist) ? READ_DATA : READ_ADD;
        end

        WRITE: ns = (SS_n == 1) ? IDLE : WRITE;
        READ_ADD: ns = (SS_n == 1) ? IDLE : READ_ADD;
        READ_DATA: ns = (SS_n == 1) ? IDLE : READ_DATA;
        default: ns = IDLE;
    endcase
end

// Output logic and counters
always @(posedge clk ) begin
    if (!rst_n) begin
        add_exist <= 0;
        counter_in <= 4'b0;
        counter_out <= 3'b0;
        rx_valid <= 0;
        rx_data <= 10'b0;
        MISO <= 0;

        start_out <= 0;
        tx_reg <= 8'b0;
    end
    else begin
        // Default assignments
        rx_valid <= 0;   // to set rx_valid to zero every clk cycle
even it was high previous to sample data correctly and dosnt coreept
with another mosi


        if (SS_n) begin
            counter_in <= 4'b0;
            counter_out <= 3'b0;
            start_out <= 0;
        end
        else begin
                 case (cs)
                IDLE: begin
                    counter_in <= 0;
                    counter_out <= 0;
                end
```

```
                WRITE :begin
                 if (counter_in < 10) begin
                   rx_data <= {rx_data[8:0], MOSI};
                  counter_in <= counter_in + 1;
                    end

                 if (counter_in == 9)
                           rx_valid <= 1;
                end
                READ_ADD: begin
                   if (counter_in < 10) begin // excute 10 time
                   rx_data <= {rx_data[8:0], MOSI};
                  counter_in <= counter_in + 1;
                    end

                 if (counter_in == 9)   // at 10th time
                           rx_valid <= 1;
                 add_exist <= 1 ;
                end
                 READ_DATA : begin
                 if (counter_in < 10) begin
                   rx_data <= {rx_data[8:0], MOSI};
                  counter_in <= counter_in + 1;
                    end

                 if (counter_in == 9)
                           rx_valid <= 1;
                 add_exist = 0 ;

                 if ( tx_valid  ) begin
                        tx_reg <= tx_data;    // save inside spi
                        start_out <= 1;
                    end
                 end
                endcase
             if (start_out && (counter_out < 8)) begin
                MISO <= tx_reg[7-counter_out];
                counter_out <= counter_out + 1;
            end
            else if (counter_out >= 8 ) begin
                start_out <= 0;
            end
        end
        end
end
endmodule
```

- **RAM:**

```
module RAM # ( parameter MEM_DEPTH = 256 , parameter ADDR_SIZE = 8)
 (
  input clk, rst_n, rx_valid,
  input [9:0] din,
  output reg  tx_valid,
  output reg [7:0] dout
);

//temporary registers for address storage
reg [ADDR_SIZE-1:0]  temp_addr_write;
reg [ADDR_SIZE-1:0]  temp_addr_read;

// Memory declaration
reg [7:0] mem [MEM_DEPTH-1:0];

always @(posedge clk) begin
  if (!rst_n) begin
    dout <= 0;
    tx_valid <= 1'b0;
  end
    else  begin
         tx_valid <= 1'b0;
         if (rx_valid) begin
       if (din[9:8] == 2'b00) begin
         temp_addr_write <= din[7:0];
       end
       else if (din[9:8] == 2'b01) begin
         mem[temp_addr_write] <= din[7:0];
       end
       else if (din[9:8] == 2'b10) begin
         temp_addr_read <= din[7:0];
       end
       else if (din[9:8] == 2'b11) begin
        tx_valid <= 1'b1;
        dout <= mem[temp_addr_read];
        end
       end
    end
 end

endmodule
```

- **SPI_Wrapper:**

```
module spi_wrapper (
    input MOSI, clk, rst_n, SS_n,
    output MISO
);

wire [9:0] rx_data;
wire [7:0] tx_data;
wire rx_valid;
wire tx_valid;

// Instantiate RAM
RAM u1 (
    .clk(clk),
    .rst_n(rst_n),
    .rx_valid(rx_valid),
    .din(rx_data),
    .tx_valid(tx_valid),
    .dout(tx_data)
);

// Instantiate SPI
spi u2 (
    .MOSI(MOSI),
    .SS_n(SS_n),
    .clk(clk),
    .rst_n(rst_n),
    .tx_valid(tx_valid),
    .tx_data(tx_data),
    .rx_valid(rx_valid),
    .rx_data(rx_data),
    .MISO(MISO)
);

endmodule
```

2. Test bench :

```
module spi_wrapper_tb ();

reg MOSI , SS_n , clk , rst_n ;
wire MISO ;
reg [7:0] write_address;

spi_wrapper dut (

     MOSI , clk , rst_n , SS_n ,
     MISO
);

initial begin
   clk = 0;
   forever #5 clk = ~clk ;
end

initial begin

    $readmemh ("mem.dat", dut.u1.mem) ;
    rst_n = 0 ;
    if ( MISO != 0 && dut.rx_valid != 0 && dut.rx_data != 10'b0) begin
        $display ("error in reset");
        $stop ;
    end
    @(negedge clk ) ;

    rst_n =  1;
    SS_n = 0 ;
     @(negedge clk ) ;
    // write adress
    MOSI = 0 ;
     @(negedge clk ) ;
     MOSI =0 ;
     @(negedge clk );
     MOSI =0 ;
     @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
```

```
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
 if (dut.rx_valid != 1)begin
        $display ("error in write address");
        $stop ;
    end
    write_address = dut.rx_data[7:0];

    $display ( "write_address =%b" , dut.rx_data[7:0]) ;  // addres
accces in the memory
    SS_n = 1;
      @(negedge clk ) ;
    SS_n = 0;
     @(negedge clk ) ;
     // write data
    MOSI = 0 ;
     @(negedge clk ) ;
      MOSI =0 ;
     @(negedge clk );
     MOSI =1 ;
     @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
    MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     if (dut.rx_valid != 1)begin
        $display ("error in write data");
        $stop ;
    end
```

```
    $display ( "write_data =%b" , dut.rx_data[7:0]) ; // data write in
the memory
    SS_n = 1;
     @(negedge clk ) ;
    SS_n = 0;
     @(negedge clk ) ;
     // read address
    MOSI = 1 ;
     @(negedge clk ) ;
      MOSI =1 ;
     @(negedge clk );
     MOSI =0 ;
     @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     if (dut.rx_valid != 1)begin
        $display ("error in read ");
        $stop ;
    end

     $display ( "read_address =%b" , dut.rx_data[7:0]) ; // address
read

    SS_n = 1;
     @(negedge clk ) ;
    SS_n = 0;
     @(negedge clk ) ;
     // reading
    MOSI = 1 ;
     @(negedge clk ) ;
      MOSI =1 ;
     @(negedge clk );
     MOSI =1 ;
     @(negedge clk );
```

```
     MOSI=0;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
    MOSI=0;
      @(negedge clk );
     MOSI=1;
      @(negedge clk );
     if (dut.rx_valid != 1)begin
        $display ("error in read ");
        $stop ;
    end
    if(dut.rx_data[7:0] != dut.u1.mem[write_address])begin
        $display("error in read data");
        $stop;
    end
     $display ( "read_data =%b" , dut.rx_data[7:0]) ; // data read
     $stop;

      end
endmodule
```

3. run.do file:

vlib work

vlog spi_wrapper.v spi_wrapper_tb.v RAM.v spi.v

vsim -voptargs=+acc spi_wrapper_tb

add wave *

add wave -position insertpoint \

sim:/spi_wrapper_tb/dut/rx_data \

sim:/spi_wrapper_tb/dut/tx_data \

sim:/spi_wrapper_tb/dut/rx_valid \

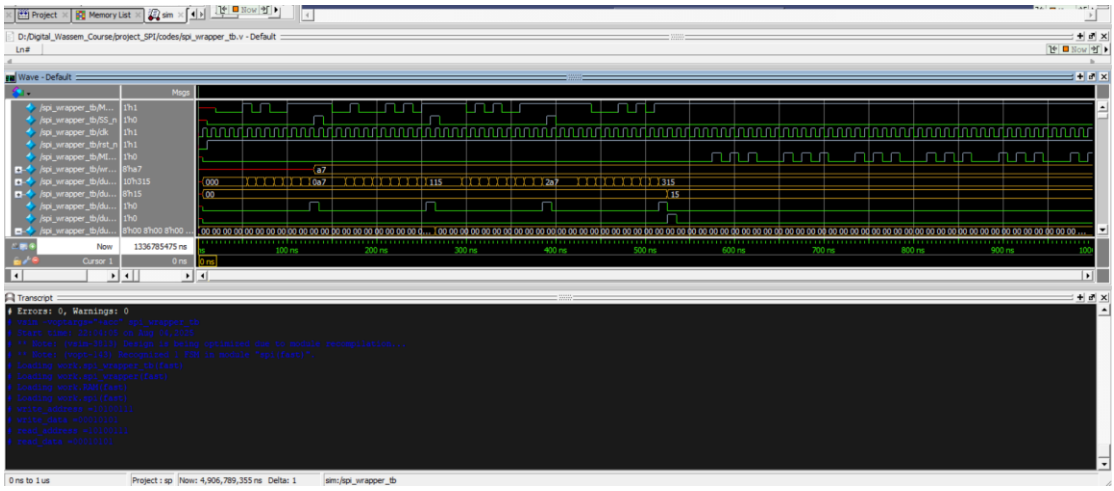
sim:/spi_wrapper_tb/dut/tx_valid \

sim:/spi_wrapper_tb/dut/u1/mem

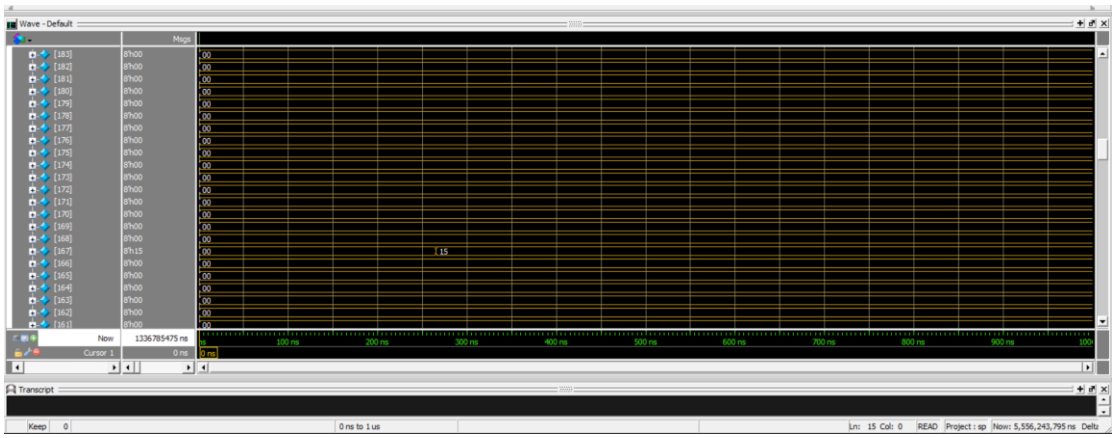
run -all

#quit -sim

4. Questa Sim snippets:



Put the value 15 at address 167



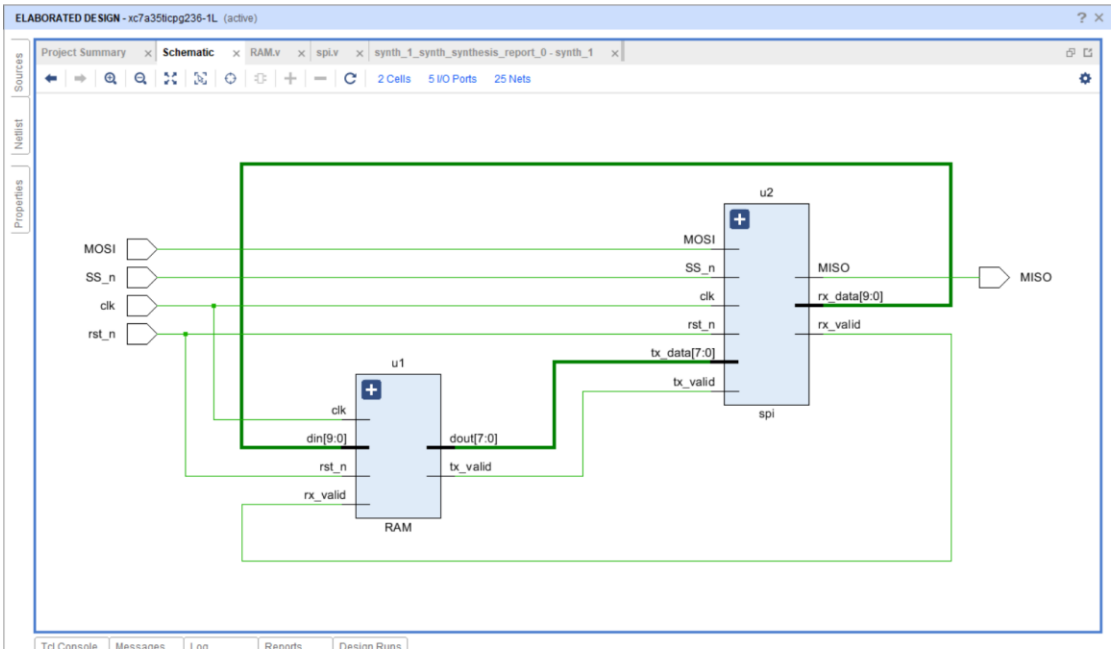
The output:

5. Constraint File:

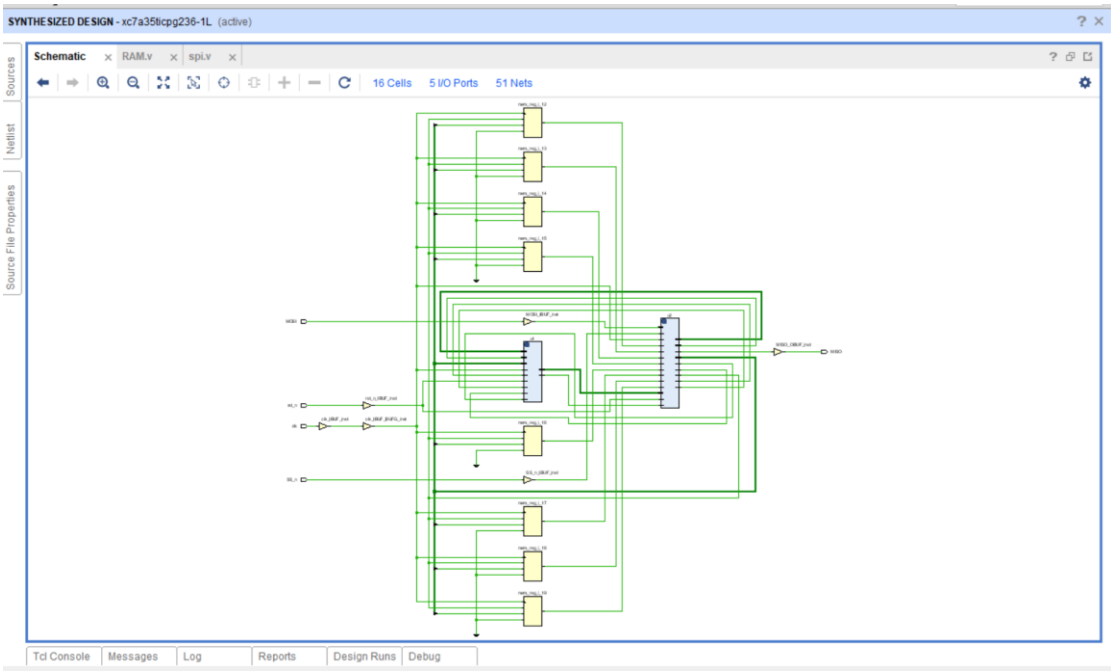
```
## Clock signal
set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports
clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5}
[get_ports clk]


## Switches
set_property -dict { PACKAGE_PIN V17   IOSTANDARD LVCMOS33 } [get_ports
{rst_n}]
set_property -dict { PACKAGE_PIN V16   IOSTANDARD LVCMOS33 } [get_ports
{SS_n}]
set_property -dict { PACKAGE_PIN W16   IOSTANDARD LVCMOS33 } [get_ports
{ MOSI}]
## LEDs
set_property -dict { PACKAGE_PIN U16   IOSTANDARD LVCMOS33 } [get_ports
{MISO}]
## Configuration options, can be used for all designs
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

## SPI configuration mode options for QSPI boot, can be used for all
designs
set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

6. Synthesis:

- **Gray Encoding**
  1. Schematic after elaboration



2. Schematic after synthesis



3. Synthesis report

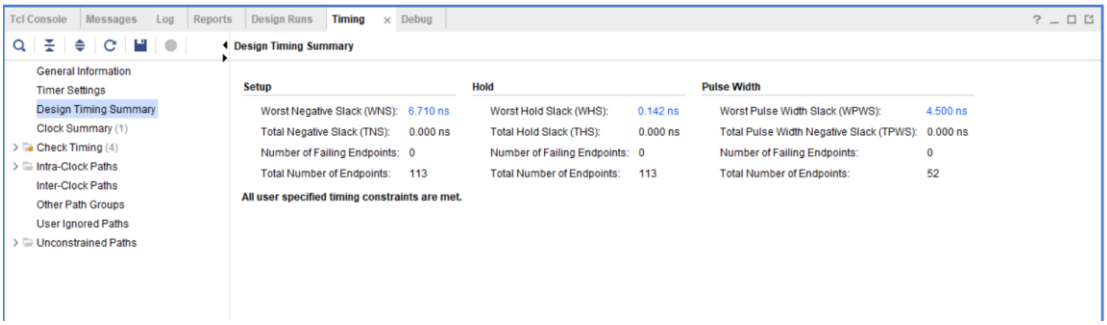
Schematic | RAM.v | spi.v | synth_1_synth_synthesis_report_0 - synth_1

D:/Digital_Wassem_Course/project_SPI/codes/viv/project_1.runs/synth_1/spi_wrapper.vds

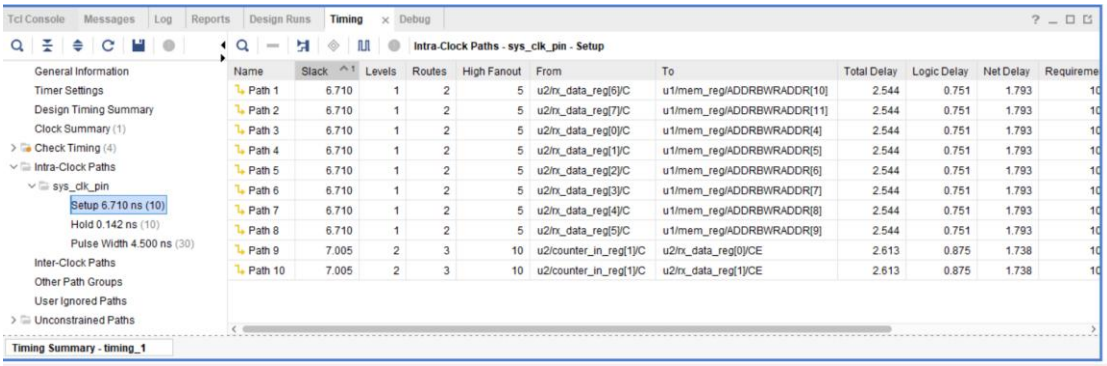
Read-only

```
97  INFO: [Synth 8-5544] ROM "ns0" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
98  INFO: [Synth 8-5544] ROM "ns0" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
99  INFO: [Synth 8-5544] ROM "ns0" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
100 INFO: [Synth 8-6430] The Block RAM mem_reg may get memory collision error if read and write address collide. Use attrib
101 ---------------------------------------------------------------------------------------------------
102                    State |                     New Encoding |                Previous Encoding
103 ---------------------------------------------------------------------------------------------------
104                     IDLE |                              000 |                             000
105                  CHK_CMD |                              001 |                             001
106                    WRITE |                              011 |                             010
107                READ_DATA |                              010 |                             100
108                 READ_ADD |                              111 |                             011
109 ---------------------------------------------------------------------------------------------------
110 INFO: [Synth 8-3354] encoded FSM with state register 'cs_reg' using encoding 'gray' in module 'spi'
111 ---------------------------------------------------------------------------------------------------
112 Finished RTL Optimization Phase 2 : Time (s): cpu = 00:00:18 ; elapsed = 00:00:20 . Memory (MB): peak = 812.984 ; gain
113 ---------------------------------------------------------------------------------------------------
114
115 Report RTL Partitions:
```

4. Timing report

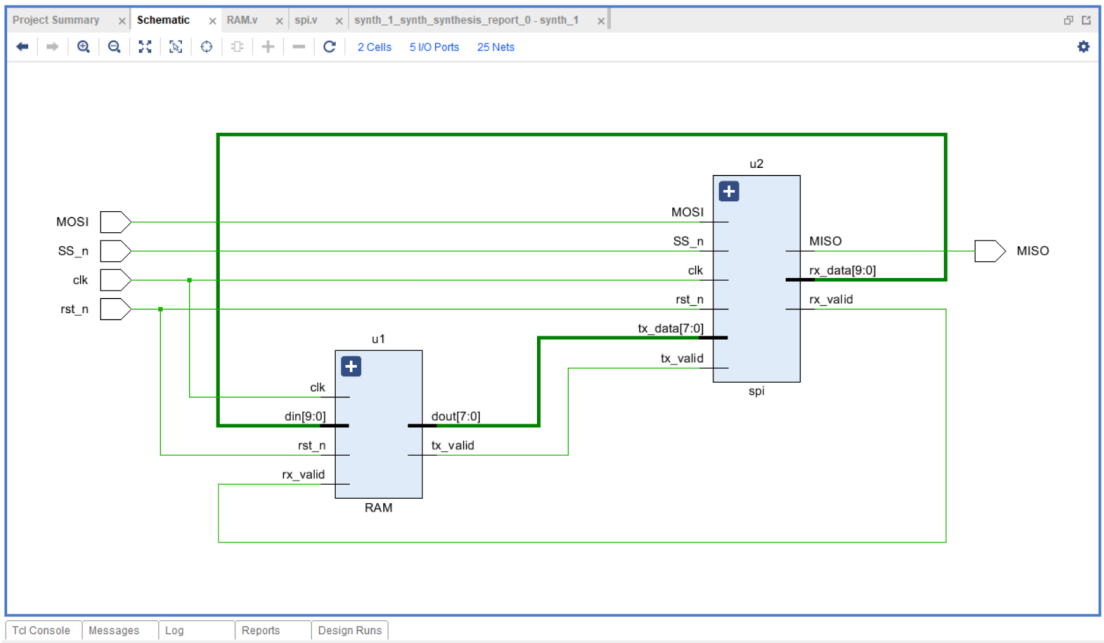


5. Critical path highlighted in the schematic

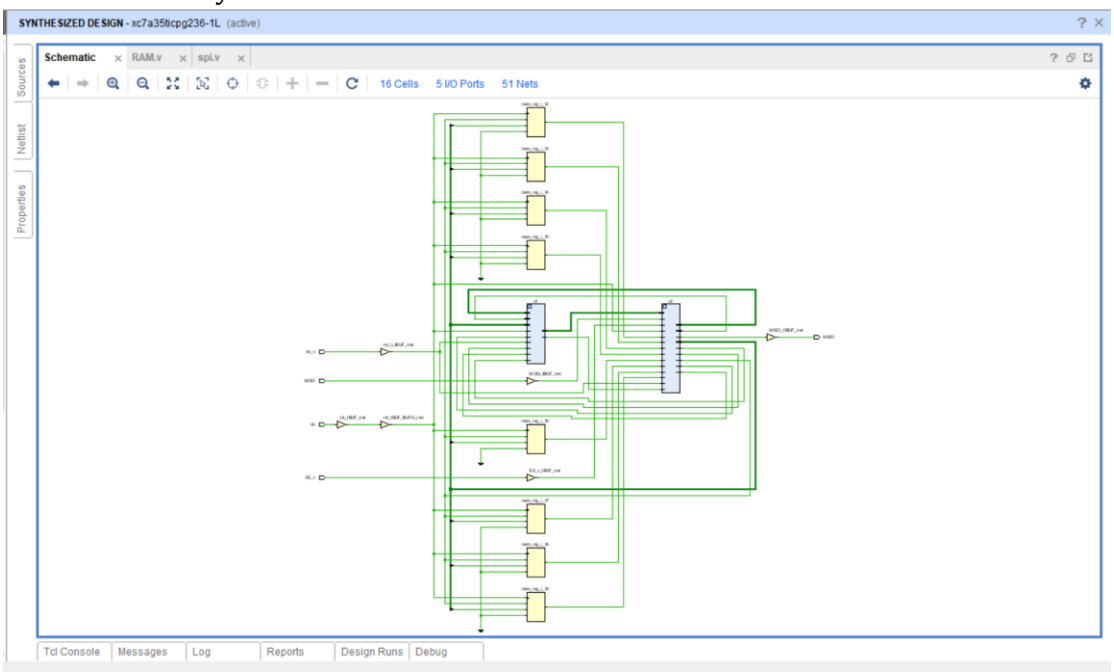


- **One_hot Encoding**

1. Schematic after elaboration



2. Schematic after synthesis



3. Synthesis rebort

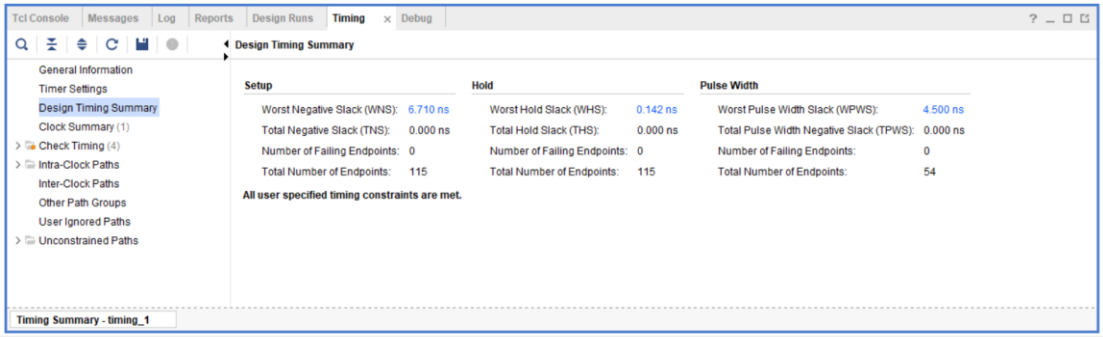
Schematic | RAM.v | spi.v | synth_1_synth_synthesis_report_0 - synth_1

D:/Digital_Wassem_Course/project_SPI/codes/viv/project_1.runs/synth_1/spi_wrapper.vds

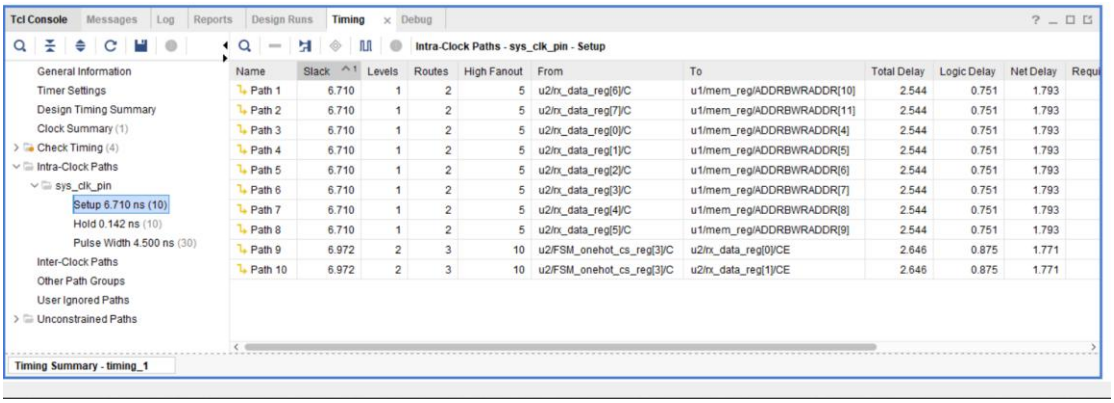
Read-only

```
INFO: [Synth 8-5544] ROM "ns0" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
INFO: [Synth 8-5544] ROM "ns0" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
INFO: [Synth 8-5544] ROM "ns0" won't be mapped to Block RAM because address size (1) smaller than threshold (5)
INFO: [Synth 8-6430] The Block RAM mem_reg may get memory collision error if read and write address collide. Use attrib
---------------------------------------------------------------------------------------------------
                   State |                     New Encoding |                Previous Encoding
---------------------------------------------------------------------------------------------------
                    IDLE |                            00001 |                             000
                  CHK_CMD |                            00010 |                             001
                   WRITE |                            00100 |                             010
               READ_DATA |                            01000 |                             100
                READ_ADD |                            10000 |                             011
---------------------------------------------------------------------------------------------------
INFO: [Synth 8-3354] encoded FSM with state register 'cs_reg' using encoding 'one-hot' in module 'spi'
---------------------------------------------------------------------------------------------------
Finished RTL Optimization Phase 2 : Time (s): cpu = 00:00:19 ; elapsed = 00:00:20 . Memory (MB): peak = 812.180 ; gain
---------------------------------------------------------------------------------------------------

Report RTL Partitions:
```

4. Timing report



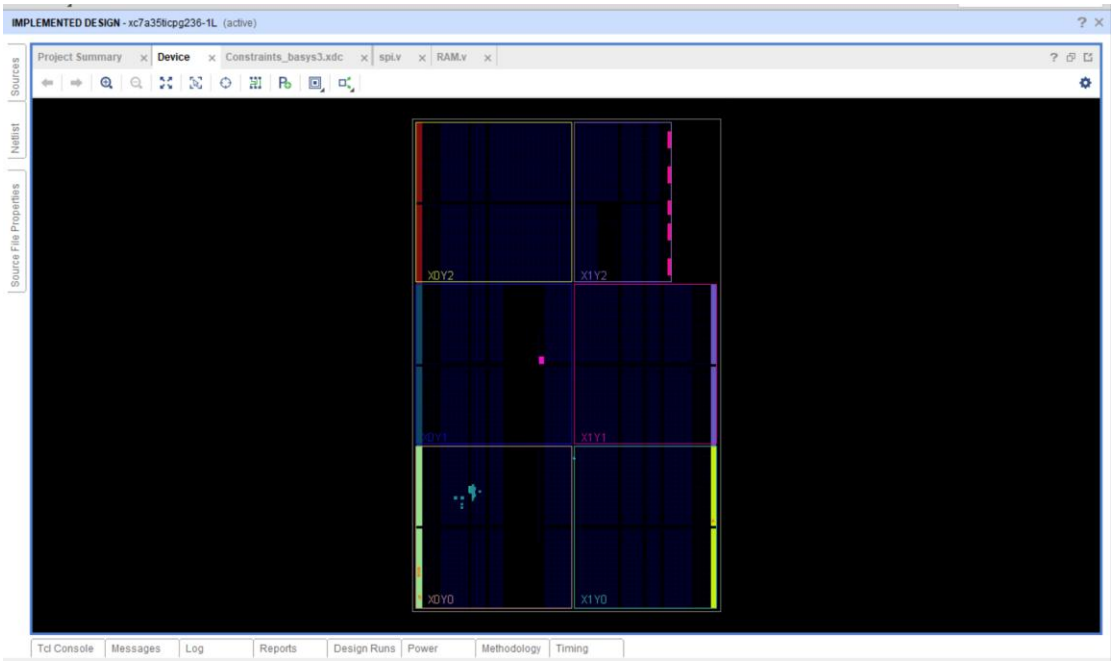
5. Critical path highlighted in the schematic



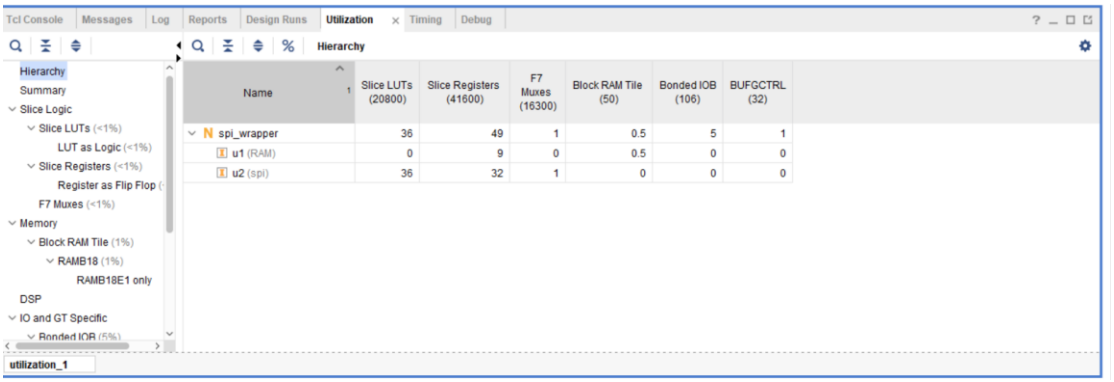
Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requ
Path 1	6.710	1	2	5	u2/rx_data_reg[6]/C	u1/mem_reg/ADDRBWRADDR[10]	2.544	0.751	1.793	
Path 2	6.710	1	2	5	u2/rx_data_reg[7]/C	u1/mem_reg/ADDRBWRADDR[11]	2.544	0.751	1.793	
Path 3	6.710	1	2	5	u2/rx_data_reg[0]/C	u1/mem_reg/ADDRBWRADDR[4]	2.544	0.751	1.793	
Path 4	6.710	1	2	5	u2/rx_data_reg[1]/C	u1/mem_reg/ADDRBWRADDR[5]	2.544	0.751	1.793	
Path 5	6.710	1	2	5	u2/rx_data_reg[2]/C	u1/mem_reg/ADDRBWRADDR[6]	2.544	0.751	1.793	
Path 6	6.710	1	2	5	u2/rx_data_reg[3]/C	u1/mem_reg/ADDRBWRADDR[7]	2.544	0.751	1.793	
Path 7	6.710	1	2	5	u2/rx_data_reg[4]/C	u1/mem_reg/ADDRBWRADDR[8]	2.544	0.751	1.793	
Path 8	6.710	1	2	5	u2/rx_data_reg[5]/C	u1/mem_reg/ADDRBWRADDR[9]	2.544	0.751	1.793	
Path 9	6.972	2	3	10	u2/FSM_onehot_cs_reg[3]/C	u2/rx_data_reg[0]/CE	2.646	0.875	1.771	
Path 10	6.972	2	3	10	u2/FSM_onehot_cs_reg[3]/C	u2/rx_data_reg[1]/CE	2.646	0.875	1.771	

7. Implementation:

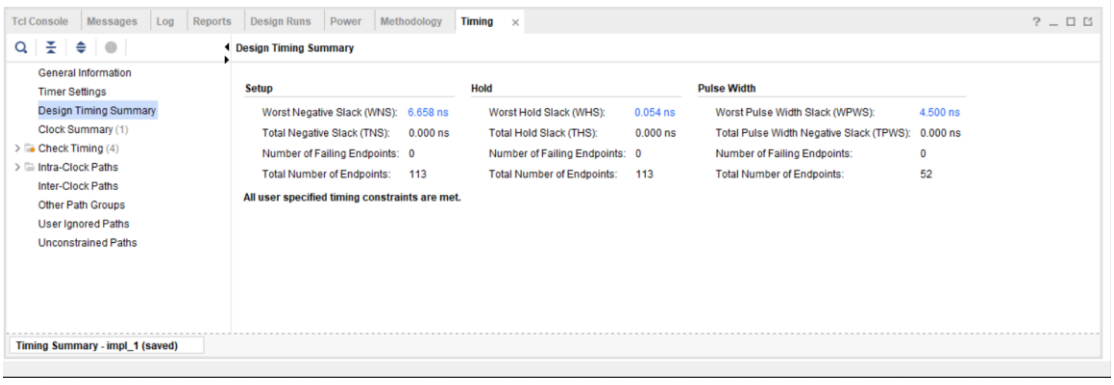
- **FPGA device**



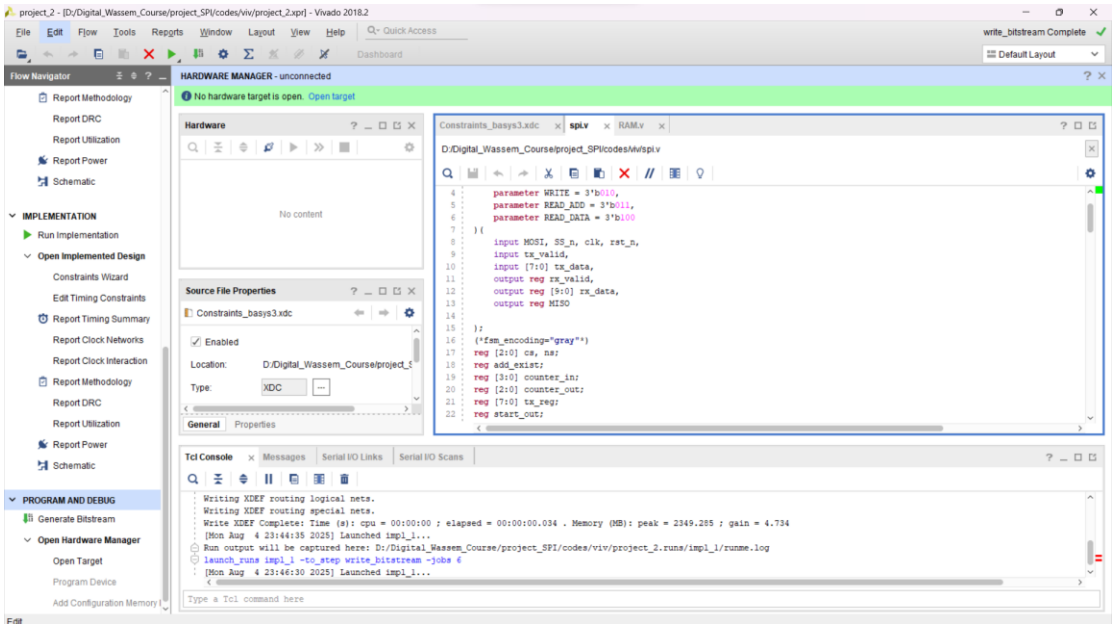
- **Utilization report**



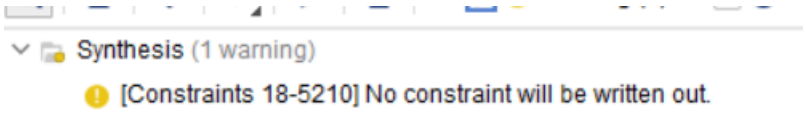
- **Timing report**



Bistream:



Messages:



8. Linting: